

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO BÀI TẬP LỚN

**ĐỀ TÀI: HỆ THỐNG QUẢN LÝ RẠP CHIẾU
PHIM THEO KIẾN TRÚC MICROSERVICE**

Giảng viên:

Kim Ngọc Bách

Nhóm lớp:

03

Nhóm bài tập:

04

Hoàng Kỳ Anh

B22DCCN016

Bùi Hoàng Sơn

B22DCCN687

Lời cảm ơn

Chúng em xin bày tỏ lòng biết ơn sâu sắc đến các Thầy/Cô giáo tại Học viện Công nghệ Bưu chính Viễn thông, cũng như các thầy cô trong Bộ môn Khoa Công nghệ thông tin, vì sự tận tâm và tâm huyết của thầy cô trong việc giảng dạy đã truyền đạt cho chúng em những kiến thức và kinh nghiệm quý báu trong suốt quá trình học tập.

Đặc biệt, chúng em chân thành cảm ơn thầy Kim Ngọc Bách, người đã nhiệt tình hướng dẫn, hỗ trợ và chỉ bảo chúng em trong suốt quá trình thực hiện dự án. Trong thời gian học tập và trao đổi cùng thầy, chúng em không chỉ thu nhận được nhiều kiến thức bổ ích mà còn học hỏi được tinh thần làm việc nghiêm túc và thái độ nghiên cứu hiệu quả, điều này sẽ rất hữu ích cho chúng em trong tương lai.

Cuối cùng, xin gửi lời cảm ơn chân thành đến thầy đã giảng dạy những kiến thức quý giá giúp chúng em trong quá trình học tập, nghiên cứu và hoàn thành báo cáo này.

Mức độ tham gia của thành viên nhóm

Họ và tên	MSSV	Nhiệm vụ	Mức độ hoàn thành
Hoàng Kỳ Anh	B22DCCN016	User Service, Cinema Service, Báo cáo	100%
Bùi Hoàng Sơn	B22DCCN687	Booking Service, Notification Service, Inventory Service, Báo cáo	100%
Đình Xuân Bách	B22DCCN052	Setup dự án(API Gateway, Eureka Server, Docker, Contract Test, Loki, Grafana)	100%

Mục lục

Mức độ tham gia của thành viên nhóm.....	3
Mục lục.....	4
DANH MỤC TỪ VIẾT TẮT.....	6
DANH MỤC HÌNH ẢNH.....	7
Chương 1. Mở đầu.....	8
1.1. Lý do chọn đề tài.....	8
1.2. Mục tiêu nghiên cứu.....	8
CHƯƠNG 2. KIẾN TRÚC VÀ THIẾT KẾ HỆ THỐNG.....	10
2.1. Kiến trúc Vi dịch vụ (Microservices Architecture).....	10
2.1.1. Tổng quan và sự chuyển dịch kiến trúc.....	10
2.1.2. Mẫu thiết kế Cơ sở dữ liệu độc lập (Database-per-service Pattern).....	13
2.1.3. Hệ thống Định tuyến và Khám phá dịch vụ (Service Discovery & API Gateway).....	15
2.2. Giao thức truyền thông và Đồng bộ dữ liệu trong hệ phân tán.....	15
2.2.1. Phân tích và lựa chọn phương thức giao tiếp.....	15
2.2.2. Bài toán nhất quán dữ liệu cuối (Eventual Consistency).....	17
2.3. Kỹ thuật Quản lý định danh và Bảo mật không trạng thái.....	18
2.3.1. Xác thực bằng JSON Web Token (JWT Stateless Authentication).....	18
2.4. Kỹ thuật Đảm bảo tính sẵn sàng và Chống chịu lỗi (Fault Tolerance).....	19
2.4.1. Vấn đề Sụp đổ dây chuyền (Cascading Failure) và giải pháp Circuit Breaker.....	19
2.4.2. Xử lý hàng đợi lỗi với cơ chế DLQ (Dead Letter Queue).....	20
CHƯƠNG 3. THỰC NGHIỆM, KIỂM THỬ VÀ ĐÁNH GIÁ HỆ THỐNG.....	21
3.1. Đặc tả API các Microservice.....	21
3.1.1. User Service APIs (/api/auth, /api/users).....	21
3.1.2. Cinema Service APIs (/api/cinemas, /api/movies, /api/showtimes).....	21
3.1.3. Booking Service APIs (/api/bookings).....	22
3.1.4. Notification Service APIs (/api/notifications).....	22
3.1.5. Inventory Service APIs (/api/products).....	23
CHƯƠNG 4. KIẾN TRÚC HƯỚNG SỰ KIỆN VÀ GIAO TIẾP BẤT ĐỒNG BỘ... 24	24
4.1. Hệ thống điều phối thông điệp (Message Broker).....	24
4.2. Đặc tả các luồng nghiệp vụ bất đồng bộ.....	27
4.2.1. Chu trình Khởi tạo đơn đặt vé (Luồng Tạo Booking).....	27
Hình 4.2: Luồng tạo đơn đặt vé (Create Booking Flow).....	27
4.2.2. Chu trình Xác nhận thanh toán (Luồng Confirm Booking).....	28
Hình 4.3: Luồng xác nhận booking sau thanh toán (Confirm Booking Flow)..	28
4.2.3. Chu trình Hủy đơn hàng (Luồng Cancel Booking).....	28

Hình 4.4: Luồng hủy đơn đặt vé (Cancel Booking Flow).....	28
CHƯƠNG 5. CƠ CHẾ BẢO MẬT VÀ TÍNH CHỊU LỖI.....	30
5.1. Xác thực và Cấp phép (Authentication & Authorization).....	30
5.1.1. Cơ chế Xác thực (Authentication).....	30
5.1.2. Cơ chế Phân quyền (Authorization - RBAC).....	30
5.2. Chống chịu lỗi với Cơ chế Ngắt mạch (Circuit Breaker).....	30
CHƯƠNG 6. CẤU HÌNH HỆ THỐNG VÀ KIỂM THỬ ĐÁNH GIÁ.....	31
6.1. Cấu hình Dịch vụ Thông báo (Notification Email Setup).....	31
6.2. Kiểm thử Hệ thống (Testing).....	31
6.2.1. Danh mục Kiểm thử Hợp đồng (Contract Tests).....	31
6.2.2. Kết quả Đánh giá.....	32
TÀI LIỆU THAM KHẢO.....	33

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Tên đầy đủ	Giải thích chi tiết
API	Application Programming Interface	Giao diện lập trình ứng dụng, cho phép các dịch vụ giao tiếp.
JWT	JSON Web Token	Phương thức xác thực không trạng thái dựa trên token JSON.
RBAC	Role-Based Access Control	Kiểm soát quyền truy cập dựa trên vai trò của người dùng.
DLQ	Dead Letter Queue	Hàng đợi lưu trữ các tin nhắn lỗi không thể xử lý trong MQ.
ACID	Atomicity, Consistency, Isolation, Durability	Các thuộc tính đảm bảo giao dịch cơ sở dữ liệu tin cậy.

DANH MỤC HÌNH ẢNH

Hình ảnh	Định nghĩa
Hình 2.1	Kiến trúc tổng quan hệ thống Microservice quản lý rạp chiếu phim
Hình 2.2	Sơ đồ Giao tiếp đồng bộ với RESTful API
Hình 2.3	Sơ đồ Giao tiếp bất đồng bộ với Message Queue
Hình 2.4	Luồng xác thực và phân quyền JWT trong hệ thống
Hình 4.1:	Luồng sự kiện qua RabbitMQ Message Broker
Hình 4.2	Luồng tạo đơn đặt vé (Create Booking Flow)
Hình 4.3	Luồng xác nhận booking sau thanh toán (Confirm Booking Flow)drawio
Hình 4.4	Luồng hủy đơn đặt vé (Cancel Booking Flow)

Chương 1. Mở đầu

1.1. Lý do chọn đề tài

Trong bối cảnh nền kinh tế số và giải trí trực tuyến đang phát triển bùng nổ, việc cung cấp trải nghiệm dịch vụ liền mạch, tốc độ cao đã trở thành yếu tố sống còn đối với các doanh nghiệp kinh doanh rạp chiếu phim. Hệ thống đặt vé không chỉ đơn thuần là nơi ghi nhận giao dịch, mà còn là trung tâm xử lý dữ liệu thời gian thực (Real-time Processing) khổng lồ.

Hiện nay, nhiều hệ thống rạp chiếu phim vừa và nhỏ vẫn đang vận hành dựa trên kiến trúc nguyên khối (Monolithic Architecture). Mặc dù dễ dàng trong giai đoạn đầu triển khai, kiến trúc này lại bộc lộ những điểm yếu chí mạng khi đối mặt với môi trường kinh doanh thực tế.

Thứ nhất là vấn đề về khả năng chịu tải và tính sẵn sàng cao. Khi có một bộ phim "bom tấn" ra mắt hoặc vào các dịp lễ Tết, lưu lượng truy cập (traffic) có thể tăng đột biến gấp nhiều lần. Với kiến trúc nguyên khối, việc một module bị quá tải sẽ làm treo cứng toàn bộ phần mềm, dẫn đến gián đoạn dịch vụ và thiệt hại trực tiếp về doanh thu.

Thứ hai là thách thức trong việc đảm bảo tính toàn vẹn và đồng bộ dữ liệu. Khi hàng ngàn người dùng cùng lúc truy cập để cạnh tranh một vị trí ghế ngồi, hiện tượng tranh chấp dữ liệu (Race Condition) rất dễ xảy ra, dẫn đến các lỗi nghiêm trọng như đặt trùng ghế (Double-booking) hoặc sai lệch giữa lượng vé bán ra và số lượng tồn kho sản phẩm phụ trợ.

Xuất phát từ thực tiễn trên, nhóm chúng em nhận thấy sự cần thiết của việc tái cấu trúc hệ thống phần mềm lõi. Đề tài "Xây dựng hệ thống quản lý rạp chiếu phim trực tuyến theo kiến trúc Microservice" ra đời nhằm giải quyết triệt để bài toán hiệu năng, hướng tới cung cấp một nền tảng đặt vé linh hoạt, có khả năng mở rộng mạnh mẽ, tính chịu lỗi cao và duy trì tính nhất quán của dữ liệu trên môi trường phân tán.

1.2. Mục tiêu nghiên cứu

Đề tài tập trung vào việc nghiên cứu và giải quyết các bài toán kỹ thuật cốt lõi trong phát triển phần mềm hiện đại, cụ thể như sau:

Về mặt lý thuyết và kiến trúc phần mềm:

- Nghiên cứu chuyên sâu về mô hình kiến trúc Microservice, hiểu rõ các mẫu thiết kế (Design Patterns) đặc thù như Database-per-service, API Gateway và Service Discovery.
- Làm chủ hệ sinh thái Spring Cloud (Eureka, Spring Cloud Gateway) để thiết lập hạ tầng định tuyến và quản lý cụm dịch vụ tự động.
- Phân tích và giải quyết bài toán chống chịu lỗi sụp đổ dây chuyền (Cascading Failure) trong môi trường phân tán bằng cơ chế ngắt mạch (Circuit Breaker) với thư viện Resilience4j.

Về công nghệ giao tiếp và đồng bộ dữ liệu:

- Phân định rõ ràng ranh giới giữa giao tiếp đồng bộ (Synchronous) thông qua RESTful API cho các tác vụ cần phản hồi tức thì, và giao tiếp bất đồng bộ (Asynchronous) cho các tác vụ nền.
- Nghiên cứu và triển khai hàng đợi thông điệp RabbitMQ (Message Broker) kết hợp với kiến trúc hướng sự kiện (Event-Driven). Xây dựng cơ chế Publish/Subscribe để tách rời luồng thanh toán, gửi thông báo Email và trừ tồn kho, giúp giảm thiểu độ trễ cho hệ thống.

Về thiết kế hệ thống cơ sở dữ liệu:

- Triển khai mô hình cơ sở dữ liệu phân tán với 5 instance MySQL hoàn toàn độc lập, xóa bỏ sự phụ thuộc lẫn nhau giữa các miền nghiệp vụ (User, Cinema, Booking, Inventory, Notification).
- Đảm bảo tính nhất quán dữ liệu cuối (Eventual Consistency) khi một giao dịch mua vé trải dài qua nhiều database khác nhau.

Về phát triển ứng dụng thực tiễn:

- Tích hợp các module nghiệp vụ thành một luồng giao dịch đặt vé – thanh toán – xuất vé hoàn chỉnh.
- Áp dụng quy trình kiểm thử hợp đồng API (Contract Testing) với JUnit 5 và Mockito để đảm bảo tính chính xác trong giao tiếp liên dịch vụ.
- Đóng gói, cấu hình môi trường triển khai bằng Docker/Docker Compose và ứng dụng Centralized Logging (Loki, Grafana) để quản lý, truy vết nhật ký hệ thống một cách trực quan, chuyên nghiệp.

CHƯƠNG 2. KIẾN TRÚC VÀ THIẾT KẾ HỆ THỐNG

Chương này tập trung trình bày các nguyên lý kỹ thuật cốt lõi và công nghệ nền tảng được áp dụng để xây dựng hệ thống Quản lý rạp chiếu phim trực tuyến. Việc nắm vững các kiến trúc phân tán, giao thức giao tiếp liên dịch vụ cũng như các giải pháp chống chịu lỗi là tiền đề quan trọng để hiểu rõ cơ chế vận hành, luồng dữ liệu và các biện pháp tối ưu hiệu năng được triển khai trong các chương tiếp theo.

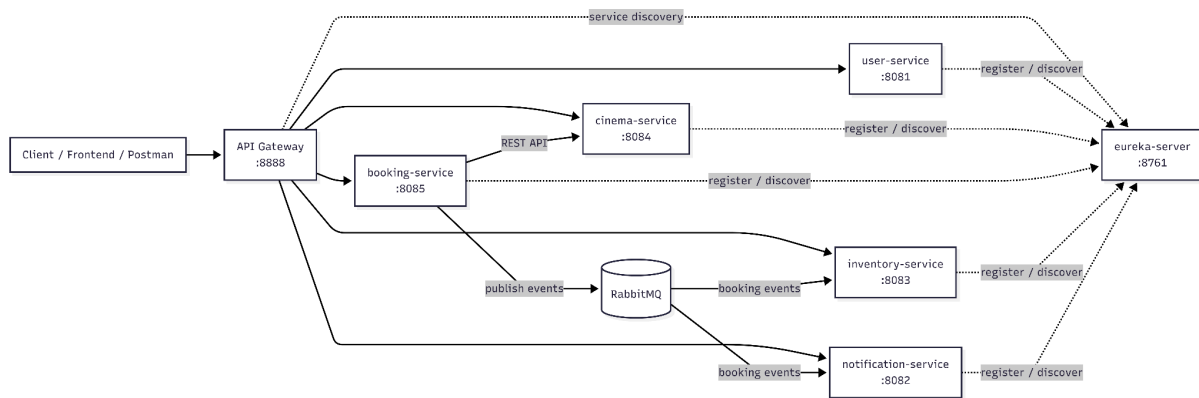
2.1. Kiến trúc Vi dịch vụ (Microservices Architecture)

2.1.1. Tổng quan và sự chuyển dịch kiến trúc

Hệ thống được thiết kế dựa trên kiến trúc Vi dịch vụ (Microservices Architecture). Trái ngược với kiến trúc Nguyên khối (Monolithic) – nơi toàn bộ giao diện, logic nghiệp vụ và truy cập cơ sở dữ liệu được đóng gói trong một mã nguồn và chạy trên một tiến trình duy nhất, Microservices phân chia ứng dụng thành một tập hợp các dịch vụ nhỏ (services).

Mỗi dịch vụ trong đồ án này (như Booking, Cinema, User) là một tiến trình độc lập, tự quản lý cơ sở dữ liệu riêng và triển khai một miền nghiệp vụ (Domain) cụ thể. Sự chuyển dịch này giải quyết triệt để bài toán "điểm nghẽn cổ chai" (Bottleneck) khi rạp chiếu phim có các đợt mở bán vé bom tấn. Thay vì phải nâng cấp phần cứng cho toàn bộ hệ thống, ta chỉ cần nhân bản (scale-out) riêng biệt Booking Service hoặc Cinema Service để đáp ứng lượng truy cập đột biến.

Sơ đồ 2.1. Kiến trúc tổng quan hệ thống



Hình 2.1: Kiến trúc tổng quan hệ thống Microservice quản lý rạp chiếu phim

2.1.2. Mẫu thiết kế Cơ sở dữ liệu độc lập (Database-per-service Pattern)

Trong các hệ thống phân tán, việc chia sẻ chung một Database vật lý sẽ phá vỡ tính độc lập của Microservice (do các service có thể khóa bảng của nhau, gây ra Deadlock). Đồ án áp dụng triệt để nguyên lý Database-per-service. Hệ thống sử dụng 5 thực thể MySQL Server hoàn toàn biệt lập (được ánh xạ qua các port 3308, 3309, 3310, 3311, 3312).

Điều này mang lại lợi ích to lớn về tính cô lập (Isolation): Nếu dịch vụ kho hàng (inventorydb) bị quá tải hoặc hỏng hóc, dịch vụ người dùng (userdb) hay dịch vụ rạp chiếu (cinemadb) vẫn hoạt động bình thường, đảm bảo khách hàng vẫn có thể đăng nhập và xem lịch chiếu.

Service	Database	Port MySQL	Các bảng chính
User Service	userdb	3311	users, user_addresses, user_payment_methods
Cinema Service	cinemadb	3308	cinemas, movies, screens, seats, showtimes
Booking Service	bookingdb	3310	bookings, booking_seats, booking_products, payments
Inventory Service	inventorydb	3309	products, cinema_inventory, stock_movements, booking_products
Notification Service	notificationdb	3312	notifications, email_templates, notification_preferences

Bảng 2.1: Phân bố cơ sở dữ liệu theo từng microservice

2.1.3. Hệ thống Định tuyến và Khám phá dịch vụ (Service Discovery & API Gateway)

Trong môi trường mạng nội bộ của Microservice, địa chỉ IP của các dịch vụ có thể thay đổi liên tục. Để giải quyết bài toán định vị, hệ thống sử dụng hai thành phần hạ tầng cốt lõi:

- Eureka Server (Service Registry): Đóng vai trò là "danh bạ" trung tâm. Khi các service khởi động, chúng tự động đăng ký tên (ví dụ: cinema-service) và địa chỉ IP hiện tại vào Eureka. Khi Booking Service cần gọi Cinema Service, nó không gọi vào IP cứng mà hỏi Eureka để lấy địa chỉ thực tế.
- Cổng - Port 8888): Mọi luồng dữ liệu từ Client (Web/Mobile) đều phải đi qua Gateway. Thành phần này đóng vai trò như một bộ định tuyến thông minh (Router) phân giải các đường dẫn. Việc này giúp ẩn giấu hoàn toàn cấu trúc mạng phức tạp phía sau khỏi Client.

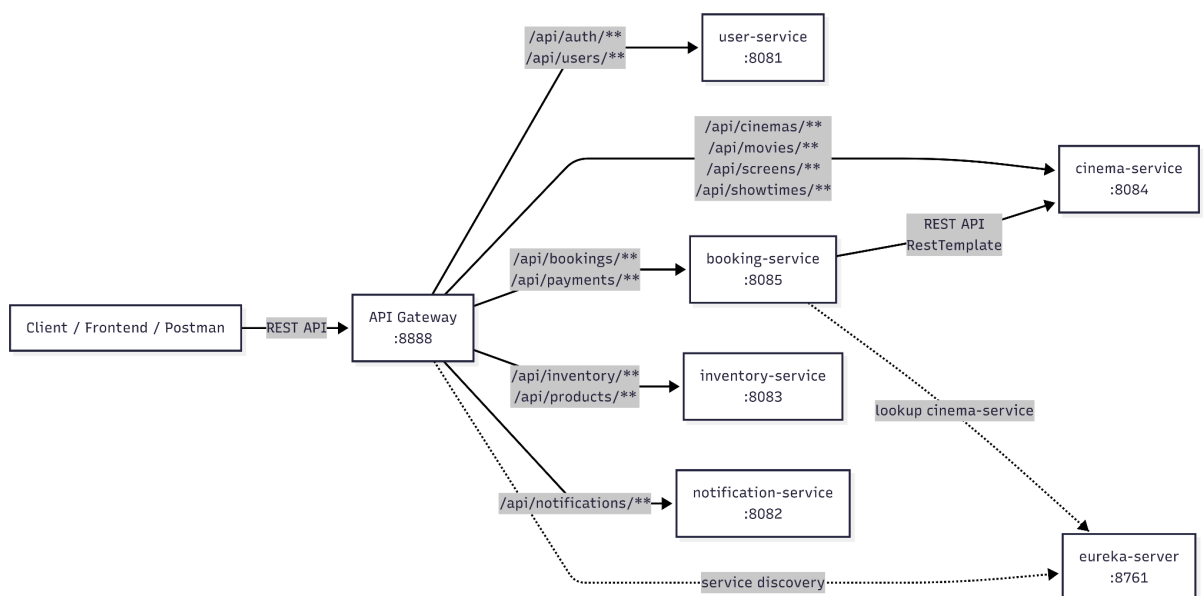
2.2. Giao thức truyền thông và Đồng bộ dữ liệu trong hệ phân tán

2.2.1. Phân tích và lựa chọn phương thức giao tiếp

Trong kiến trúc phần mềm, có hai trường phái giao tiếp chính: Đồng bộ (Synchronous) và Bất đồng bộ (Asynchronous). Thay vì chỉ dùng một loại, hệ thống kết hợp cả hai dựa trên đặc thù từng nghiệp vụ:

a) Giao tiếp đồng bộ với RESTful API (HTTP/TCP):

Được sử dụng cho các nghiệp vụ yêu cầu luồng phản hồi tức thì và bắt buộc phải biết kết quả ngay lập tức (ví dụ: Đăng nhập hệ thống, tra cứu danh sách rạp, kiểm tra ghế trống). Giao thức HTTP đảm bảo tính toàn vẹn tuyệt đối của dữ liệu. Client gửi một Request và luồng (Thread) sẽ bị chặn (block) để chờ cho đến khi nhận được Response.

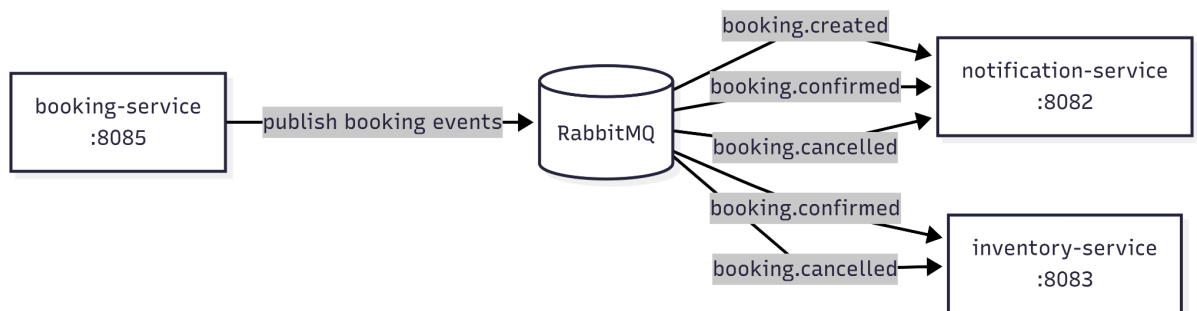


Hình 2.2: Sơ đồ Giao tiếp đồng bộ với RESTful API

b) Giao tiếp bất đồng bộ với Message Queue (RabbitMQ):

Khi một giao dịch thanh toán thành công, hệ thống phải thực hiện cùng lúc 3 việc: (1) Cập nhật trạng thái vé, (2) Gửi Email cho khách hàng, (3) Trừ số lượng bắp nước trong kho. Nếu dùng giao tiếp đồng bộ, độ trễ cộng dồn sẽ khiến màn hình đặt vé của người dùng bị "treo", dẫn đến trải nghiệm tồi tệ.

Quyết định kỹ thuật: Áp dụng RabbitMQ với nguyên lý "gửi và quên" (fire-and-forget). Booking Service chỉ cần cập nhật database nội bộ, ném một thông điệp (Message) vào hàng đợi (Queue), sau đó lập tức trả kết quả thành công cho người dùng. Các tác vụ mất thời gian như gửi Email hay tính toán trừ kho sẽ được các Consumer âm thầm xử lý ở chế độ nền (background).



Hình 2.3: Sơ đồ Giao tiếp bất đồng bộ với Message Queue

2.2.2. Bài toán nhất quán dữ liệu cuối (Eventual Consistency)

Do áp dụng cơ sở dữ liệu độc lập, hệ thống không thể sử dụng cơ chế Transaction truyền thống của các RDBMS (như ACID với Two-Phase Commit) vì nó khóa tài nguyên toàn hệ thống và làm giảm hiệu năng. Đồ án giải quyết vấn đề này bằng mô hình Hướng sự kiện (Event-Driven) để đạt được Tính nhất quán dữ liệu cuối (Eventual Consistency).

Các dữ liệu liên quan đến nhau (Vé và Kho hàng) có thể không đồng bộ tại cùng một phần nghìn giây, nhưng thông qua sự bảo đảm (durability) của RabbitMQ, nếu thông điệp booking.confirmed đã vào hàng đợi, chắc chắn inventorydb sẽ bị trừ đi số lượng tương ứng sau đó một vài mili-giây, đảm bảo hệ thống không bao giờ bị sai lệch dữ liệu về mặt tổng thể.

2.3. Kỹ thuật Quản lý định danh và Bảo mật không trạng thái

2.3.1. Xác thực bằng JSON Web Token (JWT Stateless Authentication)

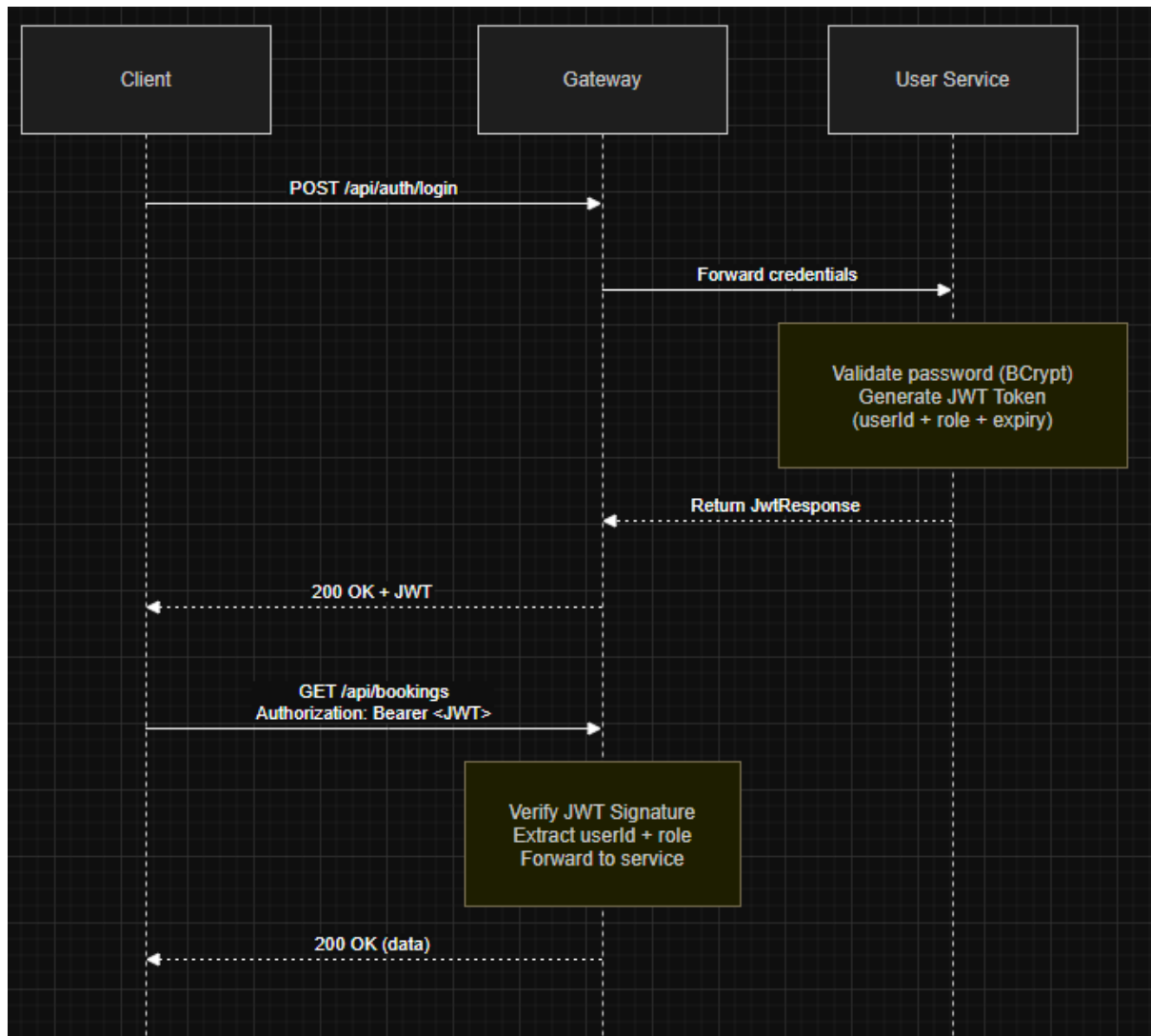
Trong mô hình nguyên khối, trạng thái người dùng thường được lưu trong Session (bộ nhớ RAM của Server). Tuy nhiên, trong Microservice, một Request thứ nhất có thể đi vào Instance A, nhưng Request thứ hai lại bị điều phối sang Instance B. Việc chia sẻ Session giữa các máy chủ là bài toán vô cùng phức tạp và tốn kém tài nguyên.

Để giải quyết bài toán trên, đồ án áp dụng cơ chế xác thực Phi trạng thái (Stateless) bằng JWT. Sau khi User Service xác thực mật khẩu qua thuật toán băm

BCrypt, nó sinh ra một chuỗi JWT chứa sẵn định danh người dùng (UserId) và Quyền hạn (Role: USER/ADMIN). Chuỗi này được ký điện tử (Cryptographic Signature) bằng một khóa bí mật (Secret Key) chỉ Server mới biết.

Server không cần lưu trữ bất kỳ thông tin nào về người dùng trên RAM. Mỗi khi Client gửi Request (có đính kèm chuỗi JWT trên Header), API Gateway chỉ cần dùng thuật toán để giải mã chữ ký. Nếu chữ ký hợp lệ, Gateway biết chắc chắn user này là ai và cho phép đi qua.

Sơ đồ 2.2. Luồng xác thực JWT



Hình 2.4: Luồng xác thực và phân quyền JWT trong hệ thống

2.4. Kỹ thuật Đảm bảo tính sẵn sàng và Chống chịu lỗi (Fault Tolerance)

2.4.1. Vấn đề Sụp đổ dây chuyền (Cascading Failure) và giải pháp Circuit Breaker

Khi người dùng đặt vé, Booking Service phải thực hiện lời gọi API đồng bộ sang Cinema Service để giữ ghế (Reserve seats). Giả sử cinemadb đang thực hiện sao lưu (backup) dẫn đến phản hồi rất chậm. Hàng ngàn luồng (Threads) của Booking Service sẽ phải đứng chờ (blocking) Cinema Service. Rất nhanh chóng, Booking Service sẽ cạn kiệt tài nguyên bộ nhớ và treo hoàn toàn, kéo theo toàn bộ hệ thống sụp đổ chỉ vì một lỗi nhỏ.

Đề án áp dụng thư viện Resilience4j để thiết lập Circuit Breaker bảo vệ các điểm gọi liên dịch vụ với các tham số cấu hình:

Tham số	Giá trị	Ý nghĩa
sliding-window-size	10	Theo dõi 10 cuộc gọi gần nhất
failure-rate-threshold	50%	Ngắt mạch khi hơn 5/10 cuộc gọi thất bại
timeout-duration	3s	Thời gian chờ tối đa mỗi cuộc gọi
wait-duration-in-open-state	10s	Thời gian chờ trước khi thử lại (HALF-OPEN)

Bảng 2.2: Cấu hình Circuit Breaker trong Booking Service

Khi cầu dao OPEN, mọi luồng gọi từ Booking sang Cinema sẽ ngay lập tức bị từ chối (Fail-fast) mà không cần chờ mạng phản hồi. Lập trình viên sẽ cung cấp một luồng dự phòng (Fallback method) để trả về thông báo: "Hệ thống rạp đang bảo trì, vui lòng đặt vé sau". Sau một khoảng thời gian, cầu dao chuyển sang trạng thái HALF-OPEN, cho phép một vài request thử nghiệm đi qua. Nếu thành công, mạch sẽ đóng lại (CLOSED), hệ thống tự phục hồi mà không cần kỹ sư can thiệp thủ công.

2.4.2. Xử lý hàng đợi lỗi với cơ chế DLQ (Dead Letter Queue)

Đối với giao tiếp bất đồng bộ, nếu Notification Service lấy Message từ Queue nhưng máy chủ SMTP của Google bị sập khiến việc gửi email thất bại, nếu không xử lý, thông báo vé của khách sẽ bị mất vĩnh viễn.

Giải pháp: Áp dụng thuật toán Retry kết hợp Dead Letter Queue. Hệ thống được cấu hình tự động thử lại 3 lần với khoảng thời gian chờ tăng dần (Exponential backoff: multiplier=2.0). Nếu quá 3 lần vẫn thất bại, RabbitMQ sẽ tự động định tuyến thông điệp này vào hàng đợi đặc biệt booking.dlq để lưu trữ an toàn. Quản trị viên có thể tra cứu các lỗi này qua API /api/notifications/failed và thực hiện gửi lại sau khi sự cố được khắc phục.

CHƯƠNG 3. THỰC NGHIỆM, KIỂM THỬ VÀ ĐÁNH GIÁ HỆ THỐNG

Chương này trình bày toàn bộ đặc tả API của các microservice, kết quả kiểm thử hệ thống và đánh giá hiệu năng tổng thể.

3.1. Đặc tả API các Microservice

3.1.1. User Service APIs (/api/auth, /api/users)

Phương thức	Endpoint	Mô tả	Phân quyền
POST	/api/auth/register	Đăng ký tài khoản mới	Public
POST	/api/auth/login	Đăng nhập, nhận JWT token	Public
GET	/api/users/me	Lấy thông tin user hiện tại	USER
PUT	/api/users/me	Cập nhật thông tin cá nhân	USER
PUT	/api/users/me/password	Đổi mật khẩu	USER
DELETE	/api/users/me	Xóa tài khoản	USER
GET	/api/users/{id}	Lấy thông tin user theo ID	ADMIN
GET	/api/users	Lấy danh sách tất cả users	ADMIN

Bảng 3.1: Đặc tả API của User Service

3.1.2. Cinema Service APIs (/api/cinemas, /api/movies, /api/showtimes)

Phương thức	Endpoint	Mô tả	Phân quyền
GET	/api/cinemas	Lấy danh sách tất cả rạp	Public
GET	/api/cinemas/{id}	Lấy chi tiết rạp	Public
POST	/api/cinemas	Tạo rạp mới	ADMIN
PUT	/api/cinemas/{id}	Cập nhật rạp	ADMIN
DELETE	/api/cinemas/{id}	Xóa rạp	ADMIN
GET	/api/movies	Lấy danh sách tất cả phim	Public
GET	/api/movies/now-showing	Phim đang chiếu	Public
GET	/api/movies/coming-soon	Phim sắp chiếu	Public
POST	/api/movies	Tạo phim mới	ADMIN
GET	/api/showtimes/movie/{id}	Suất chiếu theo phim	Public
POST	/api/showtimes/{id}/reserve	Giữ ghế tạm thời	INTERNAL

Phương thức	Endpoint	Mô tả	Phân quyền
POST	/api/showtimes/{id}/release	Giải phóng ghế	INTERNAL

Bảng 3.2: Đặc tả API của Cinema Service

3.1.3. Booking Service APIs (/api/bookings)

Phương thức	Endpoint	Mô tả	Phân quyền
POST	/api/bookings	Tạo đơn đặt vé mới	USER
GET	/api/bookings/{id}	Lấy chi tiết đơn đặt vé	USER/ADMIN
GET	/api/bookings/my-bookings	Lấy lịch sử đặt vé của user	USER
GET	/api/bookings	Lấy tất cả đơn đặt vé	ADMIN
PUT	/api/bookings/{id}/confirm	Xác nhận đặt vé sau thanh toán	USER/ADMIN
PUT	/api/bookings/{id}/cancel	Hủy đơn đặt vé	USER/ADMIN

Bảng 3.3: Đặc tả API của Booking Service

3.1.4. Notification Service APIs (/api/notifications)

Phương thức	Endpoint	Mô tả	Phân quyền
GET	/api/notifications	Xem toàn bộ lịch sử thông báo	ADMIN
GET	/api/notifications/{id}	Lấy chi tiết một thông báo	USER/ADMIN
GET	/api/notifications/failed	Truy xuất danh sách thư gửi thất bại	ADMIN
POST	/api/notifications/{id}/resend	Kích hoạt gửi lại thư bị lỗi	ADMIN
POST	/api/notifications/test-email	Kiểm tra kết nối dịch vụ SMTP	ADMIN
GET	/api/notifications/users/{userId}	Lịch sử thông báo của user	USER/ADMIN
GET	/api/notifications/preferences/users/{userId}	Cấu hình tùy chọn thông báo	USER/ADMIN
PUT	/api/notifications/preferences/users/{userId}	Cập nhật tùy chọn thông báo	USER/ADMIN
GET	/api/notifications/templates	Xem danh sách mẫu thư	ADMIN
POST	/api/notifications/templates	Thêm mới mẫu thư	ADMIN

Phương thức	Endpoint	Mô tả	Phân quyền
PUT	/api/notifications/templates/{id}	Chỉnh sửa mẫu thư	ADMIN
DELETE	/api/notifications/templates/{id}	Xóa mẫu thư	ADMIN

Bảng 3.4: Đặc tả API của Notification Service

3.1.5. Inventory Service APIs (/api/products)

Phương thức	Endpoint	Mô tả	Phân quyền
GET	/api/products	Lấy danh sách sản phẩm	Public
GET	/api/products/{id}	Lấy chi tiết sản phẩm	Public
GET	/api/products/cinema/{cinemaId}	Sản phẩm available tại rạp	Public
POST	/api/products	Tạo sản phẩm mới	ADMIN
PUT	/api/products/{id}	Cập nhật sản phẩm	ADMIN
DELETE	/api/products/{id}	Xóa sản phẩm	ADMIN

Bảng 3.5: Đặc tả API của Inventory Service

CHƯƠNG 4. KIẾN TRÚC HƯỚNG SỰ KIỆN VÀ GIAO TIẾP BẤT ĐỒNG BỘ

Trong môi trường vi dịch vụ (Microservices), việc xử lý tuần tự toàn bộ các tác vụ trong một giao dịch có thể gây ra hiện tượng nghẽn cổ chai và làm tăng độ trễ (latency). Để giải quyết bài toán này, hệ thống áp dụng kiến trúc hướng sự kiện (Event-Driven Architecture) thông qua việc sử dụng hàng đợi thông điệp (Message Queue), giúp các dịch vụ giao tiếp với nhau một cách bất đồng bộ.

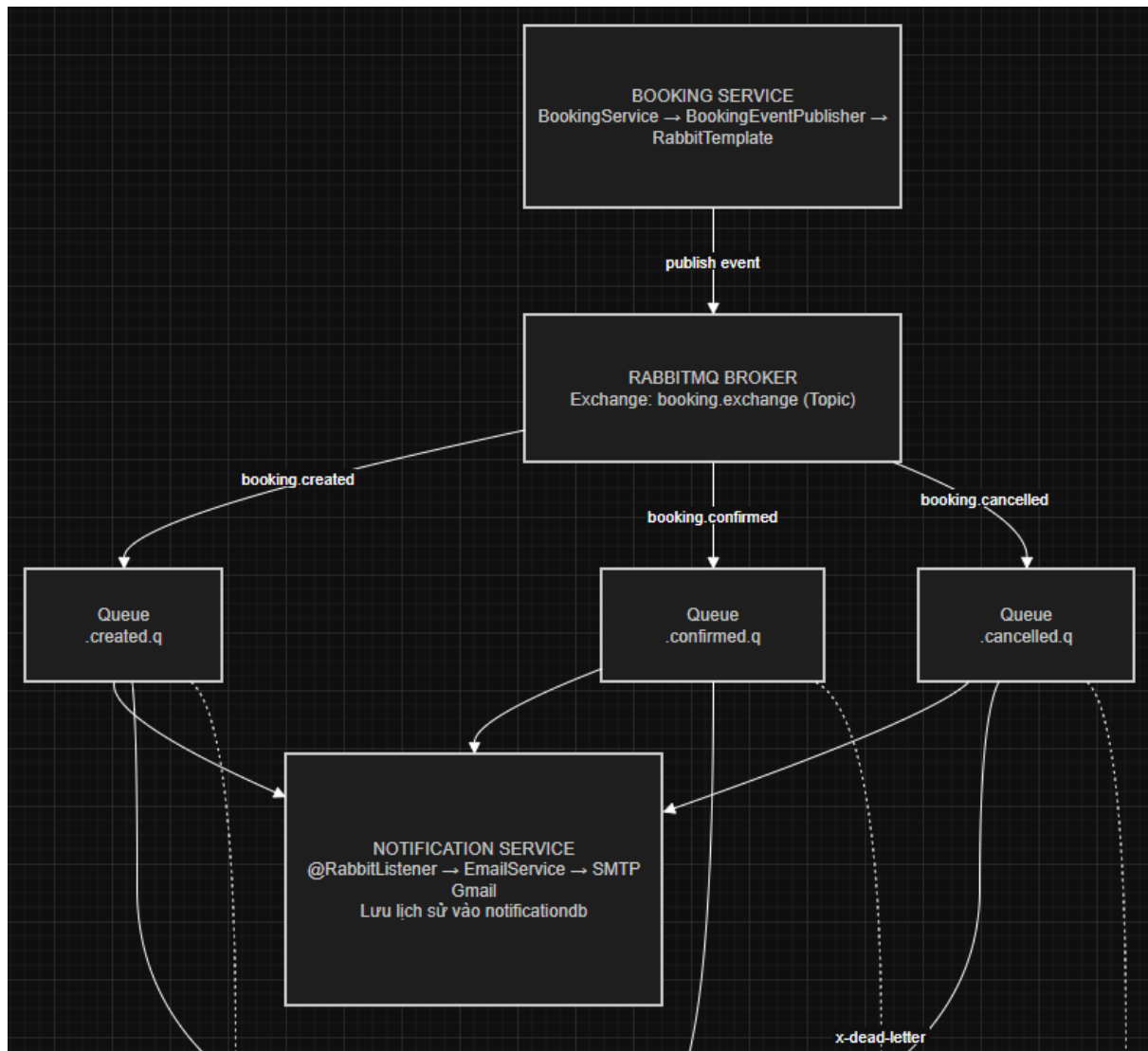
4.1. Hệ thống điều phối thông điệp (Message Broker)

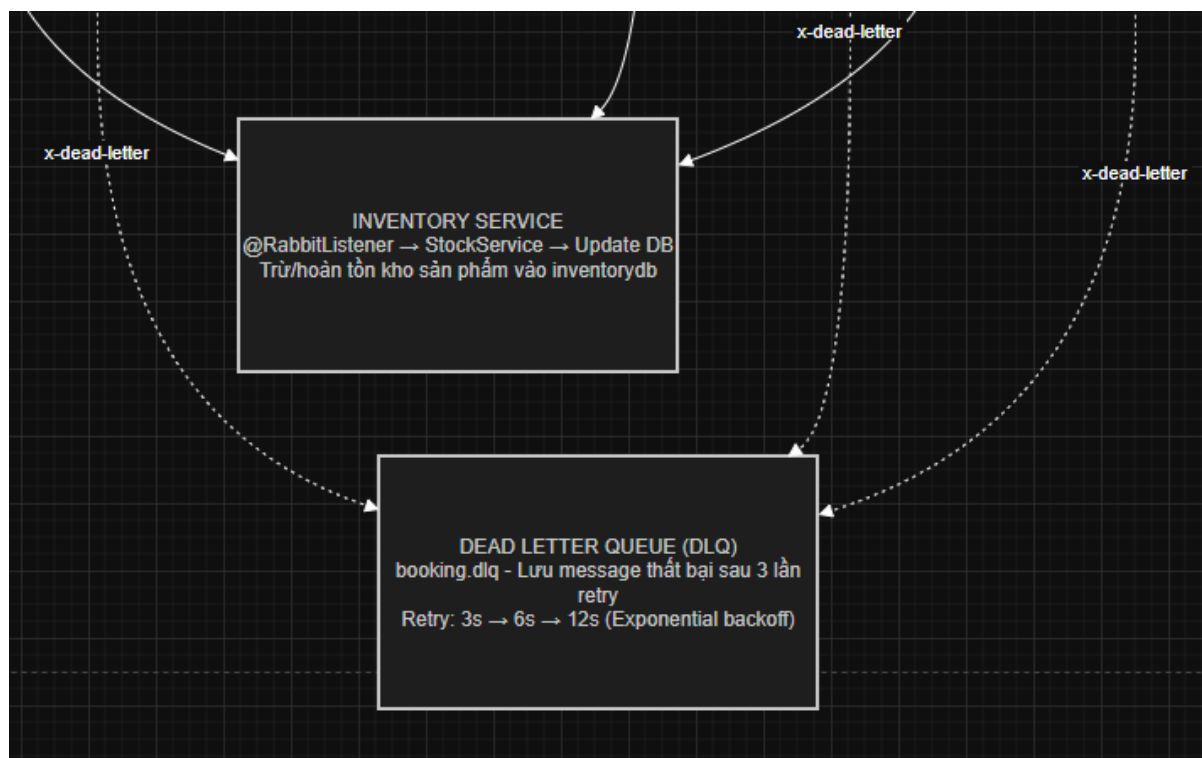
Dự án sử dụng RabbitMQ làm trung tâm điều phối thông điệp. Trong kiến trúc này, Booking Service đóng vai trò là nhà xuất bản (Publisher), chịu trách nhiệm phát ra các sự kiện khi có sự thay đổi về trạng thái của đơn đặt vé. Ngược lại, Notification Service và Inventory Service đóng vai trò là người tiêu thụ (Consumers), liên tục lắng nghe hàng đợi để thực thi các tác vụ nền.

Tên sự kiện (Event)	Routing Key	Mục đích nghiệp vụ
Booking Created	booking.created	Kích hoạt gửi Email thông báo yêu cầu người dùng thanh toán cho đơn hàng vừa khởi tạo.
Booking Confirmed	booking.confirmed	Gửi Email xác nhận xuất vé điện tử thành công, đồng thời kích hoạt luồng trừ số lượng tồn kho sản phẩm.
Booking Cancelled	booking.cancelled	Gửi Email xác nhận hủy đơn, đồng thời kích hoạt luồng hoàn trả lại số lượng tồn kho sản phẩm.

Bảng 4.1: Danh sách các sự kiện cốt lõi trong hệ thống

Sơ đồ 4.1. Luồng event RabbitMQ

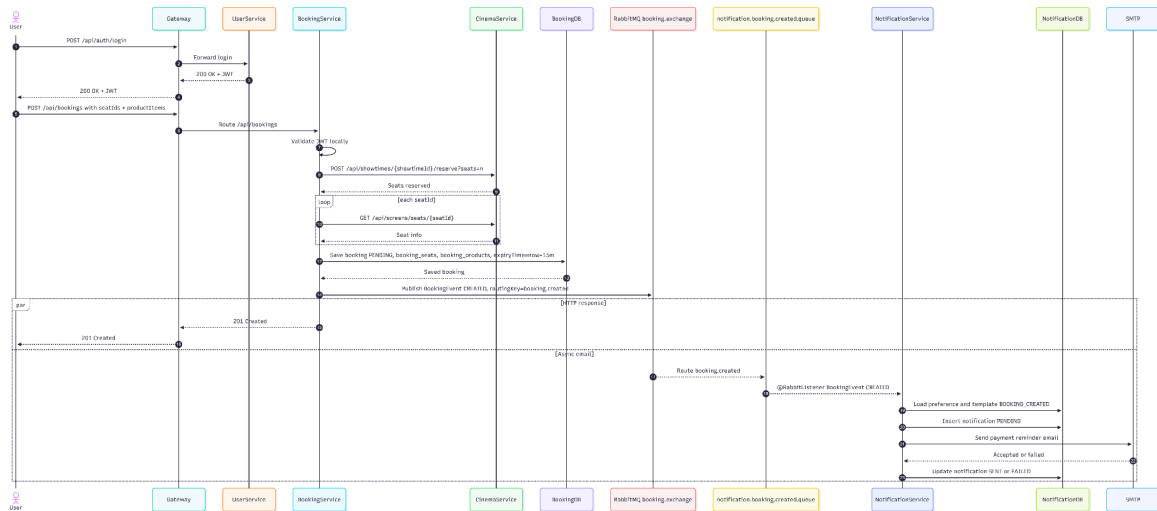




Hình 4.1: Luồng sự kiện qua RabbitMQ Message Broker

4.2. Đặc tả các luồng nghiệp vụ bất đồng bộ

4.2.1. Chu trình Khởi tạo đơn đặt vé (Luồng Tạo Booking)



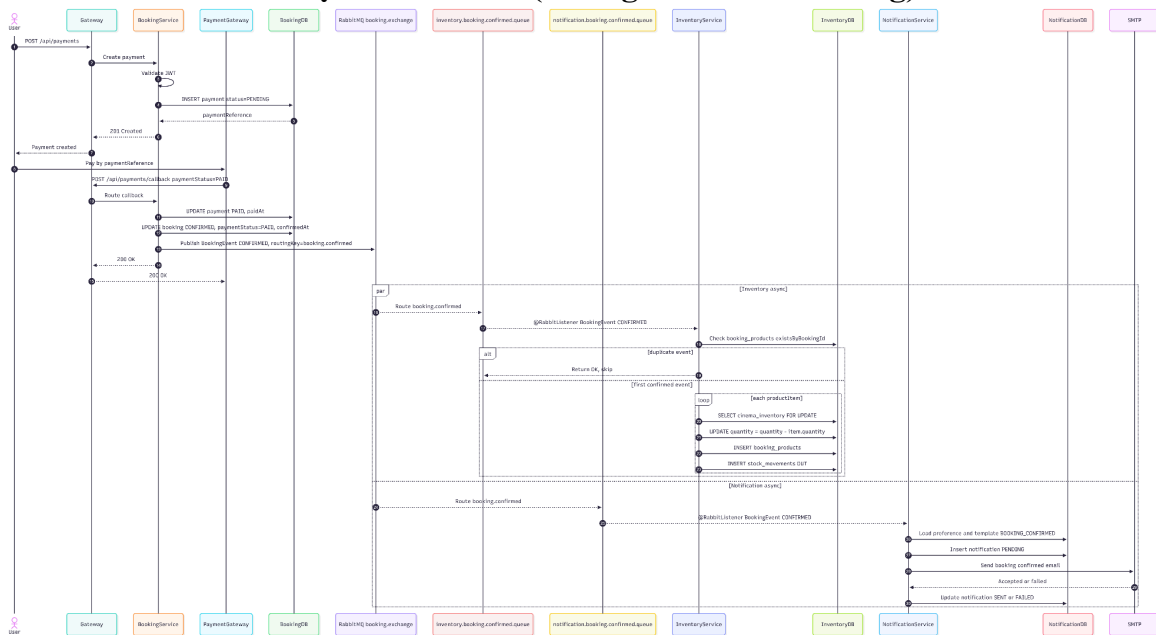
Hình 4.2: Luồng tạo đơn đặt vé (Create Booking Flow)

Mô tả chi tiết các bước:

1. User → Gateway: Gửi POST /api/auth/login.
2. Gateway → UserService: Chuyển tiếp yêu cầu đăng nhập.
3. UserService → Gateway: Trả về 200 OK kèm JWT token.
4. Gateway → User: Phản hồi 200 OK cùng JWT token.
5. User → Gateway: Gửi POST /api/bookings kèm seatIds và productItems.
6. Gateway → BookingService: Định tuyến đến /api/bookings.
7. BookingService: Xác thực JWT cục bộ.
8. BookingService → CinemaService: Gọi POST /api/showtimes/{showtimeId}/reserve?seats=n để giữ chỗ.
9. CinemaService → BookingService: Phản hồi "Seats reserved" báo giữ chỗ thành công.
10. Lập cho mỗi seatId:
 - BookingService → CinemaService: Gọi GET /api/screens/seats/{seatId}.
 - CinemaService → BookingService: Trả về thông tin ghế (Seat info).
11. BookingService → BookingDB: Ghi đơn đặt vé với trạng thái PENDING, bao gồm booking_seats, booking_products, và expiryTime = now + 15 phút.
12. BookingDB → BookingService: Xác nhận đã lưu thành công.
13. BookingService → RabbitMQ: Xuất bản sự kiện booking.created với routing key tương ứng.
14. BookingService → Gateway: Gửi phản hồi 201 Created.
15. Gateway → User: Trả lại 201 Created cho người dùng.
16. RabbitMQ → CreatedQueue: Chuyển sự kiện booking.created đến hàng đợi notification.booking.created.queue.
17. CreatedQueue → NotificationService: Gửi sự kiện đến NotificationService qua @RabbitListener.
18. NotificationService → NotificationDB: Truy vấn cấu hình và mẫu thông báo BOOKING_CREATED.

19. NotificationService → NotificationDB: Ghi bản ghi notification với trạng thái PENDING.
20. NotificationService → SMTP: Gửi email nhắc thanh toán.
21. SMTP → NotificationService: Trả về kết quả Accepted hoặc Failed.
22. NotificationService → NotificationDB: Cập nhật trạng thái notification thành SENT hoặc FAILED.
23. (Kết thúc) Toàn bộ quá trình hoàn thành sau khi email được xử lý.

4.2.2. Chu trình Xác nhận thanh toán (Luồng Confirm Booking)

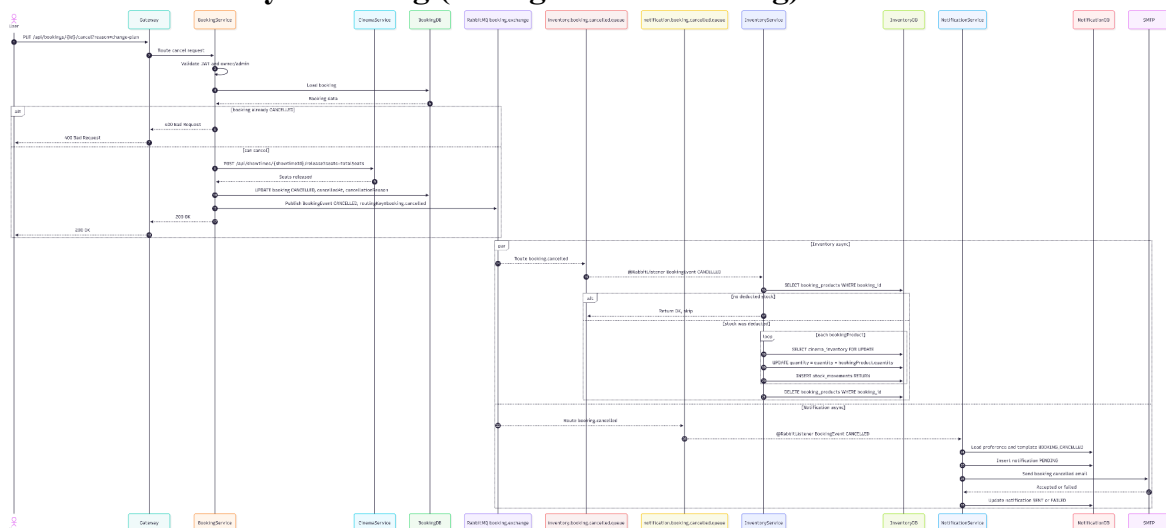


Hình 4.3: Luồng xác nhận booking sau thanh toán (Confirm Booking Flow)

Mô tả chi tiết các bước:

1. User gửi yêu cầu POST /api/payments đến Gateway để tạo giao dịch thanh toán.
2. Gateway định tuyến yêu cầu tới BookingService.
3. BookingService xác thực JWT của người dùng.
4. BookingService kiểm tra booking trong cơ sở dữ liệu hoặc bộ nhớ cache.
5. BookingService tạo bản ghi payment trong BookingDB với trạng thái PENDING.
6. BookingDB trả về thông tin paymentReference cho BookingService.
7. BookingService phản hồi 201 Created về Gateway, kèm paymentReference.
8. Gateway trả phản hồi tạo thanh toán thành công về cho User.
9. User dùng paymentReference để thanh toán qua PaymentGateway (ví dụ VNPay, Stripe).
10. PaymentGateway xử lý và xác nhận thanh toán thành công.
11. PaymentGateway gửi callback POST /api/payments/callback paymentStatus=PAID tới Gateway.
12. Gateway định tuyến callback này đến BookingService.
13. BookingService nhận callback và xác thực chữ ký hoặc mã định danh an toàn.

- ### 4.2.3. Chu trình Hủy đơn hàng (Luồng Cancel Booking)



Hình 4.4: Luồng hủy đơn đặt vé (Cancel Booking Flow)

Mô tả chi tiết các bước:

1. User gửi yêu cầu PUT /api/bookings/{id}/cancel?reason=change-plan.
2. Gateway nhận request từ người dùng.
3. Gateway định tuyến yêu cầu tới BookingService.
4. BookingService xác thực JWT và kiểm tra quyền chủ sở hữu hoặc admin.
5. BookingService truy vấn BookingDB để tải thông tin booking.
6. BookingDB trả về dữ liệu booking cho BookingService.
7. BookingService kiểm tra trạng thái booking.
8. Nếu booking đã CANCELLED, trả lỗi 400 Bad Request về Gateway.
9. Gateway phản hồi 400 Bad Request cho User.
10. Nếu booking có thể hủy, BookingService gọi CinemaService: POST /api/showtimes/{showtimeId}/release?seats=totalSeats.
11. CinemaService xác nhận ghế đã được giải phóng.
12. BookingService cập nhật BookingDB: đặt trạng thái CANCELLED, thời gian cancelledAt, và cancellationReason.
13. BookingService gửi sự kiện BookingEvent (CANCELLED) đến RabbitMQ với routing key booking.cancelled.
14. BookingService trả về 200 OK cho Gateway.
15. Gateway trả về 200 OK cho User.
16. RabbitMQ định tuyến sự kiện booking.cancelled đến InventoryQueue.
17. InventoryQueue gửi sự kiện tới InventoryService qua @RabbitListener.
18. InventoryService truy vấn InventoryDB để lấy các booking_products theo booking_id.
19. Nếu không có hàng bị trừ, InventoryService hoàn tất và bỏ qua xử lý.
20. Nếu có hàng bị trừ, lặp qua từng bookingProduct:
21. InventoryService truy vấn InventoryDB bằng SELECT FOR UPDATE để khóa dòng tồn kho.
22. InventoryService cập nhật lại số lượng tồn kho: $quantity = quantity + bookingProduct.quantity$.
23. InventoryService ghi nhận stock_movements (bản ghi hoàn hàng).
24. Sau vòng lặp, InventoryService xóa bản ghi booking_products với booking_id hiện tại.
25. RabbitMQ định tuyến sự kiện booking.cancelled đến NotificationQueue.
26. NotificationQueue gửi sự kiện tới NotificationService qua @RabbitListener.
27. NotificationService truy vấn NotificationDB để tải preference và template BOOKING_CANCELLED, chèn thông báo PENDING, gửi email qua SMTP, nhận phản hồi, và cập nhật trạng thái SENT hoặc FAILED.
28. SMTP phản hồi kết quả gửi email (đã nhận hoặc lỗi) để NotificationService cập nhật NotificationDB.

Quy tắc hủy booking: User chỉ có thể cancel booking của chính mình (hoặc ADMIN). Không thể cancel booking đã qua thời gian chiếu. Chính sách hoàn tiền (tương lai): Cancel trước 2 giờ → 100% refund, từ 30 phút đến 2 giờ → 50%, dưới 30 phút → không hoàn tiền.

CHƯƠNG 5. CƠ CHẾ BẢO MẬT VÀ TÍNH CHỊU LỖI

5.1. Xác thực và Cấp phép (Authentication & Authorization)

5.1.1. Cơ chế Xác thực (Authentication)

Bảo mật là yếu tố sống còn đối với hệ thống giao dịch thương mại. Do đặc thù phân tán, hệ thống áp dụng cơ chế bảo mật phi trạng thái (Stateless Security) thông qua JSON Web Token (JWT).

- Người dùng thực hiện đăng nhập thông qua điểm cuối `/api/auth/login`.
- Sau khi xác thực thông tin thành công, hệ thống cấp phát một chuỗi mã thông báo JWT có thời hạn 24 giờ.
- Đối với mọi API yêu cầu quyền truy cập, Client bắt buộc phải đính kèm chuỗi JWT vào Header: `Authorization: Bearer <JWT_TOKEN>`

5.1.2. Cơ chế Phân quyền (Authorization - RBAC)

Hệ thống triển khai mô hình Kiểm soát truy cập dựa trên vai trò (Role-Based Access Control - RBAC) với 2 phân quyền chính: USER và ADMIN.

Nhóm API	Vai trò	Phạm vi truy cập
Public APIs	Tất cả	Tra cứu phim, rạp, suất chiếu, sản phẩm; Đăng ký/Đăng nhập
User APIs	USER	Quản lý tài khoản cá nhân, đặt vé, xem lịch sử booking
Admin APIs	ADMIN	CRUD phim/rạp/suất chiếu, quản lý user, quản lý tồn kho, cấu hình template email

Bảng 5.1: Phân quyền truy cập theo vai trò (RBAC)

5.2. Chống chịu lỗi với Cơ chế Ngắt mạch (Circuit Breaker)

Để bảo vệ hệ thống khỏi hiệu ứng sụp đổ dây chuyền (Cascading Failure) khi một dịch vụ gặp sự cố, hệ thống tích hợp cơ chế Circuit Breaker (Cầu dao điện) tại các điểm giao tiếp liên dịch vụ rủi ro cao.

Cụ thể, booking-service áp dụng Circuit Breaker khi thực hiện các cuộc gọi đồng bộ sang cinema-service (để kiểm tra và giữ ghế). Trong trường hợp cinema-service phản hồi quá chậm hoặc liên tục báo lỗi, Circuit Breaker sẽ chủ động "ngắt mạch" (Open State). Lúc này, mọi lời gọi từ booking-service sẽ lập tức bị từ chối và trả về kết quả dự phòng (Fallback) một cách nhanh chóng, giúp giải phóng tài nguyên luồng (Thread) tại booking-service.

CHƯƠNG 6. CẤU HÌNH HỆ THỐNG VÀ KIỂM THỬ ĐÁNH GIÁ

6.1. Cấu hình Dịch vụ Thông báo (Notification Email Setup)

Notification Service được thiết kế để hỗ trợ gửi thư điện tử thực tế thông qua giao thức SMTP (Simple Mail Transfer Protocol). Dịch vụ sử dụng thư viện Spring Mail kết hợp với máy chủ SMTP của Google.

Cấu hình application.properties:

```
notification.email.enabled=true
notification.email.from=hoangky.a3th.2k4@gmail.com@gmail.com
spring.mail.host=smtp.gmail.com
spring.mail.port=587
spring.mail.username=hoangky.a3th.2k4@gmail.com@gmail.com
spring.mail.password=qyoj qoyg kovq ykww
spring.mail.properties.mail.smtp.auth=true
spring.mail.properties.mail.smtp.starttls.enable=true
```

6.2. Kiểm thử Hệ thống (Testing)

Để đảm bảo tính đúng đắn của các điểm giao tiếp (API) giữa các vi dịch vụ, dự án áp dụng phương pháp Kiểm thử Hợp đồng (Contract Testing). Phương pháp này giúp xác minh rằng các HTTP Method, Path, Status Code và Body Payload đều tuân thủ chặt chẽ đặc tả thiết kế.

6.2.1. Danh mục Kiểm thử Hợp đồng (Contract Tests)

Dịch vụ (Service)	Tập tin Kiểm thử (Test File)	Trạng thái
User Service	UserApiContractTest.java	Pass
Cinema Service	CinemaApiContractTest.java	Pass
Booking Service	BookingApiContractTest.java	Pass
Inventory Service	InventoryApiContractTest.java	Pass
Notification Service	NotificationApiContractTest.java	Pass
API Gateway	GatewayRouteContractTest.java	Pass
Eureka Server	(Kiểm thử Context Load)	Pass

Bảng 6.1: Danh mục kiểm thử hợp đồng API

6.2.2. Kết quả Đánh giá

Tiến hành thực thi bộ kiểm thử tự động thông qua lệnh biên dịch Maven (mvn -q test) trên toàn bộ các module. Kết quả ghi nhận:

Chỉ số	Kết quả
Tổng số ca kiểm thử	21 Tests
Trạng thái	100% Pass
Lỗi logic (Failures)	0
Lỗi biên dịch/thực thi (Errors)	0
Phạm vi bao phủ	Controller/API Layer

Bảng 6.2: Kết quả kiểm thử tự động

Phạm vi bao phủ của bộ kiểm thử:

- Tính trọn vẹn của Hợp đồng HTTP (Method/Path/Status/Body)
- Các API cốt lõi: Auth/User, Address, Payment Methods
- Các API rạp chiếu: Cinema, Movie, Screen, Seat, Showtime
- Các API giao dịch: Booking, Payment
- Các API kho hàng: Product, Inventory, Stock Movement
- Các API thông báo: Templates, Preferences, Resend, Test Email
- Đảm bảo API Gateway định tuyến chính xác cho toàn bộ các tiền tố (API prefixes)

TÀI LIỆU THAM KHẢO

I. Tài liệu Sách và Chuyên khảo

- [1] Chris Richardson, Microservices Patterns: With examples in Java, Manning Publications, 2018.
- [2] Sam Newman, Building Microservices: Designing Fine-Grained Systems, O'Reilly Media, 2015.
- [3] Craig Walls, Spring in Action, 6th Edition, Manning Publications, 2022.

II. Tài liệu Trực tuyến và Nền tảng (Official Documentation)

- [4] Spring Boot Reference Documentation. <https://spring.io/projects/spring-boot>
- [5] Spring Cloud Netflix (Eureka). <https://spring.io/projects/spring-cloud-netflix>
- [6] Spring Cloud Gateway. <https://spring.io/projects/spring-cloud-gateway>
- [7] Spring Security & JWT. <https://spring.io/projects/spring-security>
- [8] RabbitMQ Documentation. <https://www.rabbitmq.com/documentation.html>
- [9] Hibernate ORM. <https://hibernate.org/orm/documentation/>
- [10] Resilience4j Circuit Breaker. <https://resilience4j.readme.io/>
- [11] Grafana and Loki. <https://grafana.com/docs/>

III. Các Tiêu chuẩn và Kiến trúc Hệ thống

- [12] Microservices Architecture Best Practices. <https://microservices.io/>
- [13] The Twelve-Factor App. <https://12factor.net/>
- [14] JSON Web Token (JWT) Introduction. <https://jwt.io/introduction>